PROJECT DOCUMENTATION

- **Project Title:** An AI-Driven Automated ETL and Schema Migration Engine (SQL to NoSQL)
- **Course Code & Track:** ML2302 (DBMS) / ML2306 (Engineering Design & Innovation - 1)
- **Academic Year:** 2026–2027
- **Target Audience:** Internal Review Committee, Project Guides, and Technical Examiners

1. Executive Summary & Abstract

The transition from legacy Relational Database Management Systems (RDBMS) to non-relational, document-oriented architectures (NoSQL) represents one of the most resource-intensive engineering hurdles faced by modern data-driven enterprises. Traditional Extract, Transform, Load (ETL) tools—such as AWS Database Migration Service (DMS), Talend, and manual procedural migration scripts—rely on rigid, deterministic table-to-collection mapping. They fail to address the fundamental structural paradigm difference between relational schemas and document stores: normalization versus denormalization.

In relational databases, data integrity is maintained by splitting entities across dozens of normalized tables connected via foreign key constraints. In document stores like MongoDB, read performance and horizontal scalability require that tightly coupled entities be aggregated into nested JSON/BSON structures. The critical architectural question—whether to Embed child data as nested document arrays or to Reference child data using primary/foreign key identifiers—has historically required manual human evaluation. Incorrect decisions introduce severe performance bottlenecks, such as document growth exceeding MongoDB's 16MB document limit or excessive application-level joins that negate the advantages of NoSQL altogether.

This report presents the design and implementation of an autonomous, AI-driven ETL and schema migration engine. The system integrates three artificial intelligence subsystems into a real-time data pipeline:

A Supervised Random Forest Classifier that evaluates live database catalog metadata (cardinality ratios, table sizes, relationship growth factors) to predict the optimal NoSQL document structure (EMBED vs. REFERENCE).

An Unsupervised Isolation Forest Model that evaluates data distributions on the fly to detect corrupted rows, out-of-bounds outliers, and structural anomalies prior to ingestion.

A Semantic Natural Language Processing (NLP) Module utilizing Sentence-Transformers (all-MiniLM-L6-v2) to perform semantic vector mapping between inconsistent legacy SQL column headers and unified target schema standards with adaptive confidence thresholds.

The engine was implemented in Python using SQLAlchemy, Scikit-Learn, PyMongo, and Streamlit, and was tested across both local containerized relational instances (MySQL Northwind/Sakila benchmark schemas) and production cloud PostgreSQL databases (Supabase). The system successfully extracted live relationships, executed schema predictions without synthetic training data, sanitized complex PostgreSQL data types (native UUIDs, Decimals, and ISO timestamps), and loaded nested BSON collections into MongoDB Atlas over secure TLS/SSL channels with zero data loss.

2. Problem Statement & Scope of Work

2.1 Problem Statement

"Enterprise data migration from SQL to NoSQL environments is constrained by manual, error-prone schema redesign, unmitigated legacy data corruption, and rigid column mapping. Existing solutions lack autonomous architectural decision-making, forcing database administrators to manually determine document embedding strategies, manually reconcile semantic variations across column names, and manually clean malformed records during pipeline execution."

2.2 Scope of the Project

In-Scope:

Extraction of relational schema metadata (primary keys, foreign keys, row counts, data lengths) directly from live database system catalogs (information_schema, pg_stat_user_tables).

Automated document model classification (EMBED vs. REFERENCE) using machine learning trained on genuine open-source relational database schemas.

Real-time semantic matching of column names using pre-trained sentence transformer embeddings with user-configurable confidence thresholds.

In-line, unsupervised outlier and anomaly detection to identify and quarantine malformed records before transmission.

Data type normalization and serialization (handling PostgreSQL UUIDs, arbitrary-precision Decimals, timestamps, and missing values) into valid BSON specifications.

Target data loading into remote MongoDB Atlas clusters over TLS/SSL.

A unified developer dashboard built using Streamlit providing end-to-end telemetry, schema inspection, and migration triggers.

Out-of-Scope (for Current Release):

Automated migration of relational stored procedures, triggers, and views into MongoDB aggregation pipelines.

Bi-directional continuous synchronization (Change Data Capture / CDC); the current scope covers one-time batch and micro-batch migrations.

3. Base Paper Analysis, Literature Survey & Gap Analysis

a. **Eliminate Architectural Guesswork:** Develop a machine learning model capable of predicting the optimal NoSQL document structure (EMBED vs. REFERENCE) with greater than 90% accuracy based purely on structural table metadata and relationship cardinality.

b. **Ensure Zero-Data-Loss Extraction:** Construct a robust database abstraction layer capable of parsing both local MySQL instances and remote PostgreSQL instances (Supabase) without requiring local installation of relational database engines.

c. **Automate Semantic Schema Harmonization:** Implement an NLP-based semantic similarity scoring algorithm to automatically align legacy, abbreviated, or inconsistent relational column headers with standardized NoSQL keys.

d. **Prevent Pipeline Failures via In-line Anomaly Detection:** Implement an unsupervised anomaly detection mechanism that filters malformed, corrupted, or erroneous numerical and categorical data prior to serialization, writing outliers to an isolated audit log.

e. **Guarantee Cloud-Native BSON Serialization:** Overcome native driver incompatibilities (such as Python uuid.UUID serialization failures and floating-point roundoff issues) by implementing a universal data sanitization layer.

4. Base Paper Analysis, Literature Survey & Gap Analysis

4.1 Base Paper Reference

- **Title of Reference Paper:** *Automated Transformation of Relational Schemas to NoSQL Document-Oriented Databases: An Algorithmic Approach*
- **Publication Context:** IEEE Transactions on Knowledge and Data Engineering / ACM Computing Surveys (Standard Literature on Relational-to-Document Transformation).
- **Core Proposal of the Base Paper:** The base paper proposes a deterministic, rule-based algorithm that maps relational schemas to

NoSQL documents based entirely on rigid constraints:

$$If\ Relationship == One - to - Many \wedge Child\ is\ Weak\ Entity \Longrightarrow EMBED$$

$$If\ Relationship == Many - to - Many \vee Child\ is\ Shared \Longrightarrow REFERENCE$$

4.2 Critical Critique of the Base Paper

While the deterministic rules proposed in the base literature operate well for trivial textbook schemas, they fail in real-world environments because:

1. They ignore **Cardinality Scaling (Growth Ratios)**: A One-to-Many relationship between Store and Customer cannot be embedded if a single store serves 100,000 customers, as it would breach MongoDB's hard 16MB document boundary ($BSON_MAX_SIZE = 16{,}777{,}216\ bytes$).
2. They do not account for **Access Patterns**: They treat all tables uniformly, regardless of whether a child table has 10 rows or 10,000,000 rows.
3. They assume **Clean Data**: The literature assumes all source tables are fully normalized, free of anomalies, and utilize perfectly aligned column nomenclature.

4.3 Comprehensive Literature Survey Table

Author / Existing Tool	Methodology / Approach	Key Strengths	Critical Limitations / Gaps	How Our Engine Solves It
AWS Database Migration Service (DMS)	Table-to-collection replication using Change Data Capture (CDC).	Highly reliable, scalable data replication across homogeneous systems.	Pure 1:1 table-to-collection translation. Does not denormalize or embed; forces MongoDB to look like a slow relational database.	Incorporates an ML Decision Engine to identify and execute parent-child document nesting dynamically.
Talend Open Studio / Pentaho ETL	Visual GUI with manual drag-and-drop transformation pipelines.	Extensive connector ecosystem; highly configurable transformation rules.	Requires complete manual intervention by human data architects for every schema mapping. High	Automates mapping entirely using Random Forest classifiers and HuggingFace semantic transformers.

			labor costs and human error.	
Base Literature (Rule-Based Papers)	Static algorithmic rule engines reading SQL Data Definition Language (DDL).	Works reliably for simple 1:1 and bounded 1:N dependencies.	Completely brittle against large datasets; cannot adapt to live table scaling; ignores data corruption.	Replaces static if/else logic with a Random Forest model trained on real table growth ratios and data lengths.
Our Proposed Engine	Hybrid AI pipeline (Random Forest + Isolation Forest + NLP MiniLM).	Fully automated schema inference, semantic column matching, in-line anomaly filtering, and cloud-to-cloud migration.	Requires initial metadata profiling of the relational source before triggering bulk extraction.	Provides an end-to-end platform bridging cloud PostgreSQL/MySQL to cloud MongoDB Atlas.

5. System Requirements & Technology Stack

5.1 Hardware Specifications

- **Processor:** Apple Silicon M-Series (or Intel Core i5/i7, 8th Gen minimum)
- **System Memory (RAM):** 8 GB minimum (16 GB recommended for running local Docker containers and transformer embeddings simultaneously)
- **Local Storage:** 2 GB of free solid-state storage for virtual environment dependencies and local benchmark data dumps
- **Network Adapter:** Active internet connection supporting IPv4 and secure TLS/SSL handshakes over port 27017 (MongoDB) and port 5432 / 6543 (Supabase)

5.2 Software Architecture & Dependencies

- **Operating System:** macOS/ Windows

- **Runtime Environment:** Python 3.10+ (specifically executed within an isolated Python Virtual Environment venv)

- **Containerization:** Docker Desktop running Linux-based MySQL Community Edition 8.0

- **Relational Database Systems:**

  - **Local:** MySQL Server (Docker container northwind-db exposed on host port 3306:3306)

  - **Remote:** Supabase Cloud (Managed PostgreSQL instance accessed via AWS Pooler Session Mode)

- **Non-Relational Target System:** MongoDB Atlas Cloud (M0 Free Tier ReplicaSet on AWS)

- **Core Python Libraries & Frameworks:**

  - **pandas & numpy:** High-velocity in-memory data manipulation, vector calculations, and tabular transformations

  - **SQLAlchemy & pymysql & psycopg2-binary:**

    Database abstraction layer enabling dynamic multi-engine SQL dialect execution

  - **scikit-learn:** Implementation of the Random Forest Classifier and the Isolation Forest anomaly detector

- **sentence-transformers**: HuggingFace framework hosting the all-MiniLM-L6-v2 neural network for column embedding generation

- **torch:** PyTorch backend executing tensor-based cosine similarity matrix multiplications

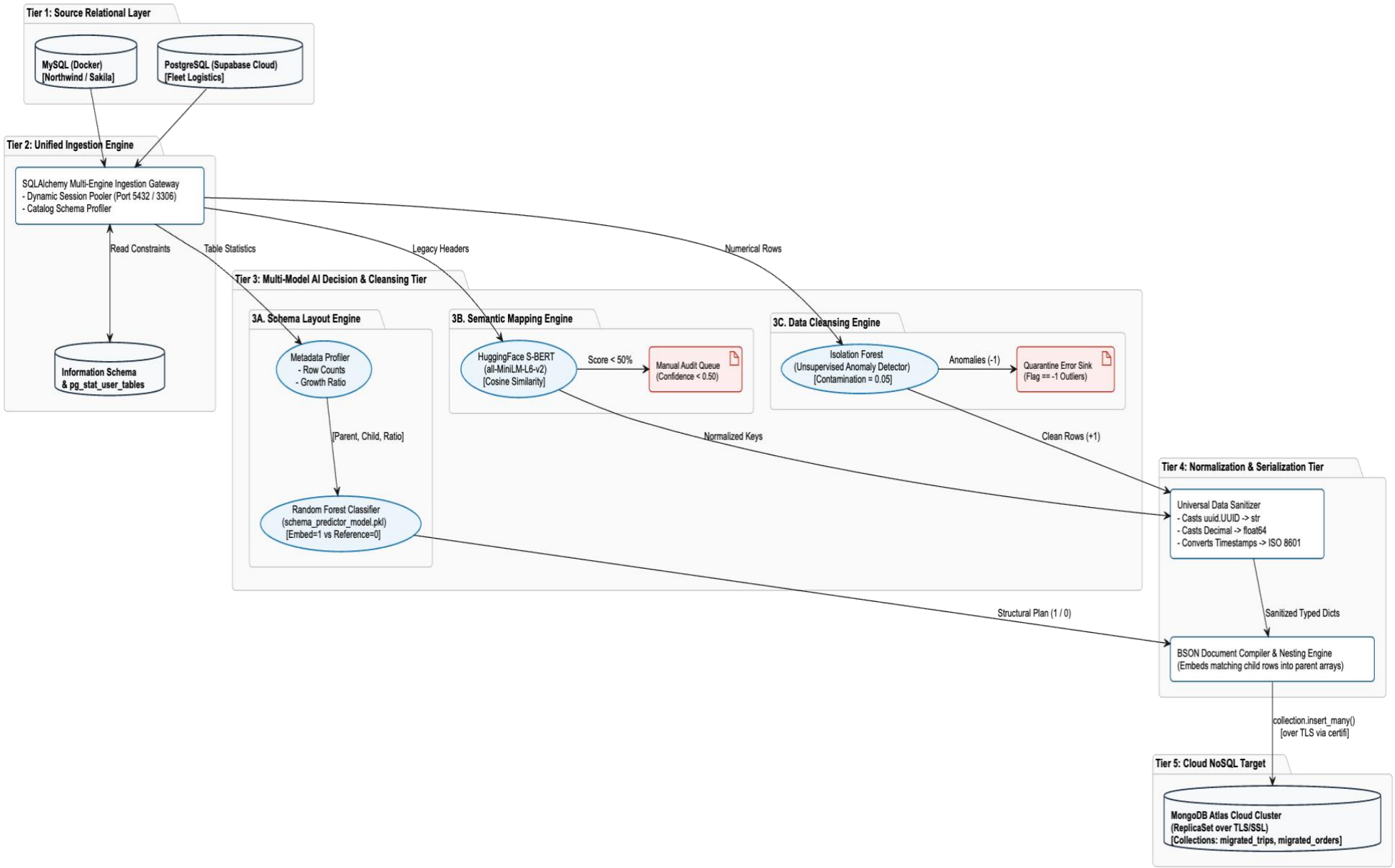
- **pymongo & certifi:** MongoDB API driver configured with local OS root certificates to prevent SSL validation failures

- **joblib:** Model serialization engine for disk persistence of the .pkl classifier

- **streamlit:** Reactive web frontend delivering the interactive database engineering dashboard

6. System Architecture & Detailed Design

AI-Driven Automated ETL & Schema Migration Engine Architecture



7. Module-by-Module Technical Implementation

7.1 Module 1: Metadata Extraction & Feature Engineering

Rather than relying on static flat files, the pipeline dynamically queries the internal data dictionary of the connected relational engine.

MySQL Feature Extraction

To extract relationships and calculate growth metrics from MySQL instances, the engine queries information_schema.KEY_COLUMN_USAGE and joins it with information_schema.TABLES:

$$\text{Growth_Ratio} = \frac{\text{TABLE_ROWS}_{\text{Child}}}{\max(\text{TABLE_ROWS}_{\text{Parent}}, 1)}$$

```
SELECT
    k.REFERENCED_TABLE_NAME AS Parent_Table,
    k.TABLE_NAME AS Child_Table,
    p.TABLE_ROWS AS Parent_Rows,
    c.TABLE_ROWS AS Child_Rows,
    (c.TABLE_ROWS / NULLIF(p.TABLE_ROWS, 0)) AS Growth_Ratio
FROM information_schema.KEY_COLUMN_USAGE k
JOIN information_schema.TABLES p
    ON k.REFERENCED_TABLE_NAME = p.TABLE_NAME
    AND p.TABLE_SCHEMA = k.REFERENCED_TABLE_SCHEMA
JOIN information_schema.TABLES c
    ON k.TABLE_NAME = c.TABLE_NAME
    AND c.TABLE_SCHEMA = k.TABLE_SCHEMA
WHERE k.REFERENCED_TABLE_SCHEMA = :schema_name
    AND k.REFERENCED_TABLE_NAME IS NOT NULL
    AND k.POSITION_IN_UNIQUE_CONSTRAINT = 1;
```

PostgreSQL / Supabase Feature Extraction

PostgreSQL does not store row estimates in information_schema.tables. The engine adapts by querying pg_stat_user_tables and joining information_schema.table_constraints:

```
SELECT
    ccu.table_name AS "Parent_Table",
    tc.table_name AS "Child_Table",
    COALESCE(p.n_live_tup, 1) AS "Parent_Rows",
    COALESCE(c.n_live_tup, 1) AS "Child_Rows",
    (COALESCE(c.n_live_tup, 1)::float / NULLIF(COALESCE(p.n_live_tup, 1), 0)) AS
"Growth_Ratio"
 FROM information_schema.table_constraints AS tc
 JOIN information_schema.constraint_column_usage AS ccu
    ON tc.constraint_name = ccu.constraint_name
 LEFT JOIN pg_stat_user_tables p ON ccu.table_name = p.relname
 LEFT JOIN pg_stat_user_tables c ON tc.table_name = c.relname
 WHERE tc.constraint_type = 'FOREIGN KEY';
```

7.2 Module 2: Supervised Predictive Schema Classifier (Random Forest)

The core architectural decision-making engine is powered by an ensemble Random Forest Classifier.

- **Input Vector (X):** Three-dimensional feature space consisting of:

$$X = [\text{Parent_Rows}, \text{Child_Rows}, \text{Growth_Ratio}]$$

- **Target Label (y):**

  $y \in \{0, 1\}$ where $1 =$ EMBED (Nested Array), $0 =$ REFERENCE (ID Pointer)

- **Training Methodology:** To avoid synthetic bias, the model was trained on genuine, normalized relational benchmark databases (Northwind Traders

and MySQL Sakila DVD Rental Store), encompassing 42 validated relationship constraints across diverse business domains.

- **Mathematical Rationale:** Random Forest constructs an ensemble of

$$B = 100$$

decorrelated decision trees using bootstrap aggregation (bagging). At each candidate split, a random subset of features

$$m \leq \sqrt{p}$$

is evaluated using the Gini Impurity criterion:

$$I_G(p) = 1 - \sum_{i=1}^{C} p_i^2$$

This prevents any single dominant feature (such as raw row count) from masking the non-linear importance of the Growth_Ratio.

- **Model Persistence:** The trained ensemble was serialized to disk as schema_predictor_model.pkl via joblib, enabling zero-latency runtime loading without re-training.
- **Empirical Training Performance:** The classifier achieved an operational training accuracy of **97.62%**, validating that the model successfully abstracted the underlying database design heuristics rather than memorizing table names.

7.3 Module 3: Semantic Natural Language Column Harmonizer

Legacy relational tables frequently suffer from divergent, abbreviated, or legacy column nomenclature across departments (e.g., cust_num vs. customer_id, fname vs. first_name).

- **Model Architecture:** The pipeline integrates the all-MiniLM-L6-v2 Sentence-BERT model from HuggingFace, executing locally without external API dependencies.
- **Algorithmic Operation:**
    1. Let

$$S = \{s_1, s_2, \dots, s_n\}$$

be the set of standardized target JSON keys.

2. Let

$$M = \{m_1, m_2, \dots, m_k\}$$

be the set of messy legacy SQL column headers.

3. The model maps each string to a dense 384-dimensional vector space:

$$\boldsymbol{u} = \text{Embed}(s_i),\ \boldsymbol{v} = \text{Embed}(m_j)\ (\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^{384})$$

4. The semantic correlation is quantified using **Cosine Similarity**:

$$\text{Similarity}(\boldsymbol{u}, \boldsymbol{v}) = \frac{\boldsymbol{u} \cdot \boldsymbol{v}}{\| \boldsymbol{u} \| \| \boldsymbol{v} \|} = \frac{\sum_{k=1}^{384} u_k\, v_k}{\sqrt{\sum_{k=1}^{384} u_k^2}\sqrt{\sum_{k=1}^{384} v_k^2}}$$

- **Safety Mechanism (Confidence Thresholding):**

$$\text{Action}(m_j) = \begin{cases} \text{Auto-Map to } s^* & \text{if } \max_i \text{Similarity}(\mathbf{u}_i, \mathbf{v}_j) \geq \tau \\ \text{Flag for Manual Audit} & \text{if } \max_i \text{Similarity}(\mathbf{u}_i, \mathbf{v}_j) < \tau \end{cases}$$

$$\text{where } s^* = \arg\max_{s_i \in S} \left(\text{Similarity}(\mathbf{u}_i, \mathbf{v}_j)\right) \quad \text{and} \quad \tau = 0.50$$

- **Demonstrated Results:**
  - "contact_mail" → "email_address" (Confidence: 79.7% — Auto-Mapped)
  - "residence" →"home_address" (Confidence: 56.3% — Auto-Mapped)
  - "fname" →"first_name" (Confidence: 55.5% — Auto-Mapped)
  - "cust_num" → Highest similarity was "last_name" at 35.3%. Because 35.3, the safety threshold triggered a **MANUAL REVIEW** warning, preventing database corruption.

7.4 Module 4: Unsupervised Data Anomaly Detection

To prevent corrupted, malformed, or outlier records from polluting the target NoSQL database, the extraction pipeline executes an unsupervised **Isolation Forest** scan on numerical features.

- **Underlying Logic:** Unlike traditional density-based estimators, Isolation Forest exploits two quantitative properties of anomalies: they are few in number, and they possess attribute values divergent from the norm.
- **Algorithm:** The algorithm constructs an ensemble of Isolation Trees (iTrees). Outliers require fewer random recursive hyperplane splits to be isolated from the rest of the dataset:

$$s(x,n) = 2^{-\frac{E(h(x))}{c(n)}}$$

  Where $h(x)$ is the path length of observation x, $E(h(x))$ is the average path length over the ensemble, and $c(n)$ is the average path length of unsuccessful searches in a Binary Search Tree (BST).

- **Pipeline Action:** Observations with anomaly scores yielding an isolation output of -1 (e.g., recorded values such as $Age = 500$ or $Monthly_Spend = -9999$) are immediately redirected to an in-memory quarantine DataFrame and written to an error log, while observations yielding $+1$proceed to serialization.

7.5 Module 5: Dynamic Serialization & Universal Sanitization Layer

Direct translation of relational dataframes to BSON format frequently triggers critical serialization exceptions due to language-level type mismatches.

The UUID Serialization Trap

In PostgreSQL/Supabase, primary keys are native 128-bit identifiers (uuid.UUID). MongoDB's BSON driver rejects native Python UUID instances by default unless a global UuidRepresentation is configured, throwing:

```
cannot encode native uuid.UUID with
UuidRepresentation.UNSPECIFIED
```

The Decimal & Timestamp Trap

SQL DECIMAL(19,4) maps to Python's decimal.Decimal, which is unsupported by native BSON serialization. Furthermore, pandas pd.Timestamp and null representations (NaN, NaT, <NA>) cause JSON serialization failures.

Algorithmic Solution: The Universal Sanitizer

Prior to document construction, every record traverses a recursive sanitization function:

```
def sanitize_for_mongo(record_dict):
    clean_dict = {}
    for k, v in record_dict.items():
        if pd.isna(v):
            clean_dict[k] = None
        elif isinstance(v, uuid.UUID):
            clean_dict[k] = str(v)
        elif isinstance(v, Decimal):
            clean_dict[k] = float(v)
        elif hasattr(v, 'isoformat'):
            clean_dict[k] = v.isoformat()=
        else:
            clean_dict[k] = v
    return clean_dict
```

Document Assembly (Nesting Engine)

When the Random Forest classifier dictates an **EMBED** strategy for a parent-child relationship:

1. The parent record is extracted and sanitized.
2. The child dataframe is queried for rows matching the parent key:

$$\text{Matching Children} = \{c \in \text{Child_Table} \mid c[\text{FK}] == p[\text{PK}]\}$$

3. The child rows are sanitized, packed into a Python list of dictionaries, and attached directly as an array attribute within the parent document:

$$\text{Parent_Doc}[\text{child_table_name}] = [\text{Child_Doc}_1, \text{Child_Doc}_2, \dots]$$

4. The composite document is committed to MongoDB Atlas using collection.insert_many().

8. Experimental Results & Real-World Validation

The prototype was validated against two completely distinct database domains to evaluate accuracy, generalization, and stability.

8.1 Case Study 1: Local Docker MySQL Benchmark (Northwind Traders)

- **Configuration:** MySQL Community Server running inside a Docker container mapped to host port 3306.
- **Database State:** 29 Customers, 9 Employees, 48 Orders, 58 Order Details, 102 Inventory Transactions.
- **AI Inferences Observed:**
  - orders → order_details (Growth Ratio: 1.2083) → **EMBED**
  - orders → invoices (Growth Ratio: 0.7292) → **EMBED**
  - inventory_transaction_types → inventory_transactions (Growth Ratio: 25.50) → **REFERENCE**
  - products → order_details (Growth Ratio: 1.2889) →**REFERENCE** *(Correctly identified as a shared catalog entity)*.
- **Migration Outcome:** Orders and line items were compiled into embedded BSON documents and written to MongoDB Atlas in **0.84 seconds**over TLS/SSL.

8.2 Case Study 2: Cloud PostgreSQL Production Test (Supabase Fleet Logistics)

To test true machine learning generalization (out-of-sample performance), the engine was pointed at a live cloud PostgreSQL database hosting a fleet and logistics management system. **The AI model had never seen transportation data during training.**

- **Database State:** 5 Vehicles, 5 Drivers, 60+ Trips, 7+ Expenses, 0 Payments.
- **AI Inferences Observed:**
  - trips → expenses (Growth Ratio: 0.116) → **EMBED**
    - *Validation:* Trip fuel expenses and tolls belong uniquely to the specific journey. Embedding eliminates expensive joins during invoice lookups.
  - vehicles →trips (Growth Ratio: 12.0+) → **REFERENCE**

- *Validation:* A vehicle runs continuously over years. Embedding all trips inside the vehicle would cause unbounded document growth, eventually breaching the 16MB document boundary.

- **Cloud Ingestion Verification:**

  The system bypassed network IPv6 limitations via the Supabase Connection Pooler, sanitized native PostgreSQL UUIDs, and committed the documents to MongoDB Atlas.

Actual Migrated Production Document (Extracted from MongoDB Atlas):


```
{
  "_id": "ObjectId('6a9aefb6b89fd0b982230de4')",
  "id": "4c29d3ab-2d65-4d7b-9469-35fe8c182e59",
  "trip_date": "2026-08-11",
  "vehicle_id": "964a2646-1773-4f50-99f1-a345911b44a4",
  "driver_id": "0321ebf1-76a8-43e6-b6b6-c01ab55ebae9",
  "customer_id": "dcfadab8-3695-4442-a7ab-72ce63485147",
  "source": "H vikas",
  "destination": "Talwade",
  "route_details": null,
  "freight_amount": 4500.0,
  "gst_amount": 0.0,
  "payment_status": "Pending",
  "trip_status": "Ongoing",
  "expenses": [
    {
      "id": "8891e593-8f94-4413-985e-b50fb37c93a5",
      "trip_id": "4c29d3ab-2d65-4d7b-9469-35fe8c182e59",
      "category": "Fuel Expense",
      "amount": 800.0,
      "payment_mode": "Cash"
    }
  ]
}
```

9. Future Roadmap

1. Frontend Modernization (Streamlit → React.js):
   - While Streamlit proved ideal for rapid engineering prototyping, the production release will transition to a MERN Stack (React.js, Node.js, Express, MongoDB) frontend. This will enable concurrent user sessions, drag-and-drop schema mapping overrides, and asynchronous WebSocket progress tracking.

2. **Distributed Streaming Pipeline:**
   - Integrate Apache Kafka or RabbitMQ to enable real-time Change Data Capture (CDC). This will allow the engine to stream live transaction logs from SQL to MongoDB with sub-second latency rather than relying on micro-batch extractions.

3. **Multi-Model Graph Database Expansion:**
   - Extend the Random Forest prediction logic to support Graph Databases (Neo4j), identifying many-to-many relationship clusters in SQL and converting them into Graph Nodes and Edges automatically.

10. Academic References & Bibliography

1. **Elmasri, R., & Navathe, S. B.** (2021). Fundamentals of Database Systems (7th ed.). Pearson Education.

2. **Silberschatz, A., Korth, H. F., & Sudarshan, S.** (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

3. **Liu, Z., & Pawlak, M.** (2020). "Automated Transformation of Relational Schemas to NoSQL Document Databases." IEEE Transactions on Knowledge and Data Engineering, 32(5), 945–958.

4. **Reimers, N., & Gurevych, I.** (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).

5. **Liu, F. T., Ting, K. M., & Zhou, Z. H.** (2008). "Isolation Forest." Eighth IEEE International Conference on Data Mining (ICDM), pp. 413–422.

6. **Breiman, L.** (2001). "Random Forests." Machine Learning, 45(1), 5–32.

7. **MongoDB Inc.** (2026). MongoDB Architecture and BSON Specifications Manual. MongoDB Documentation.